



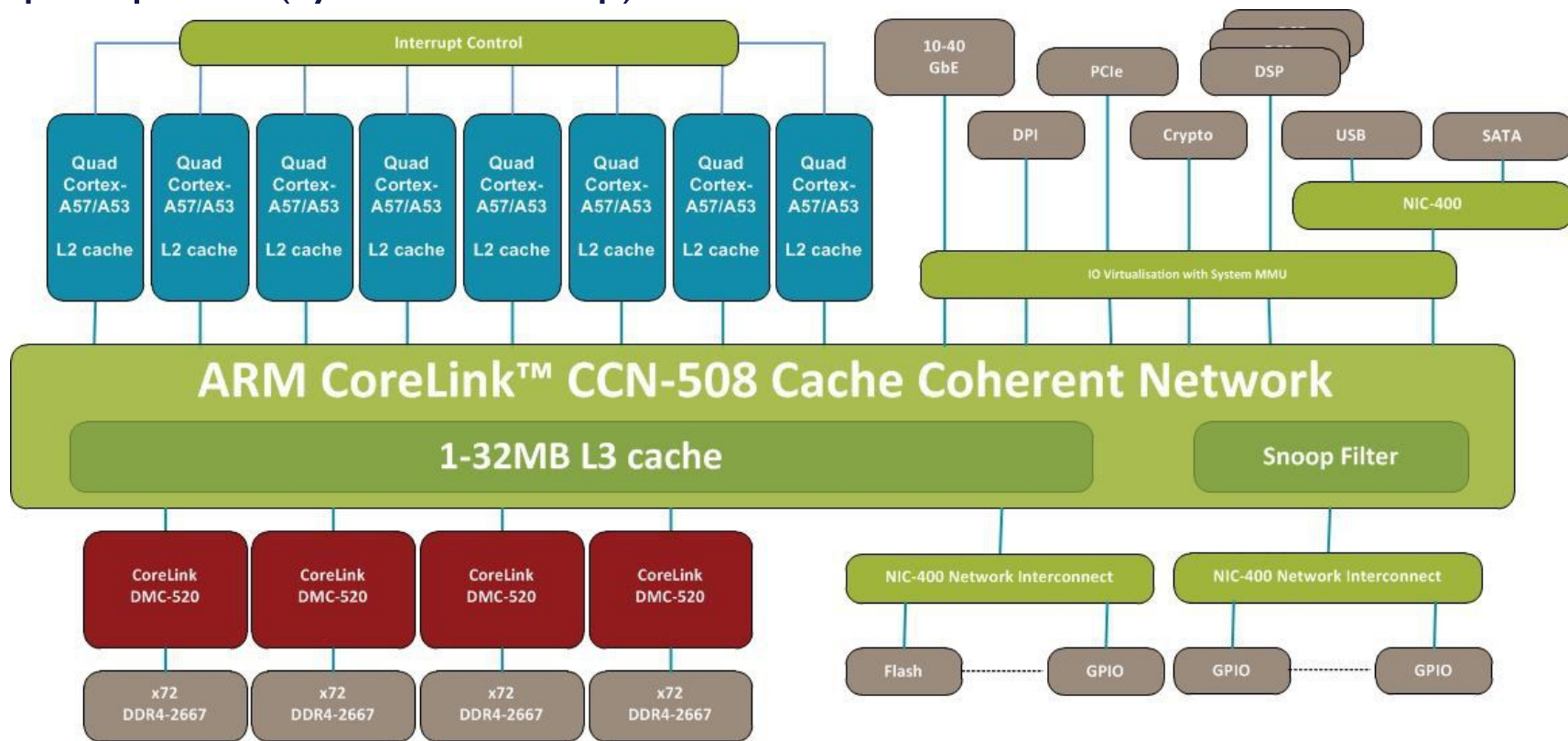
Управление потоком данных

Проектирование цифровых вычислительных систем

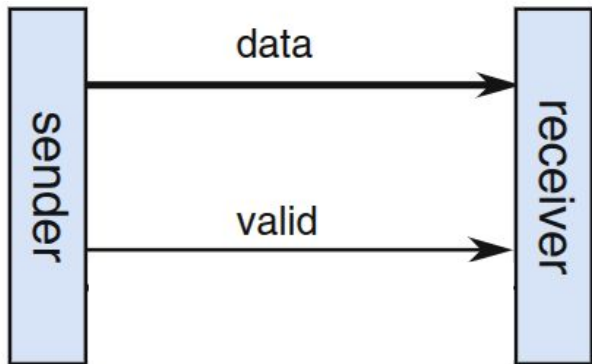
Прутьянов В. В.



Пример SoC (System-on-Chip)

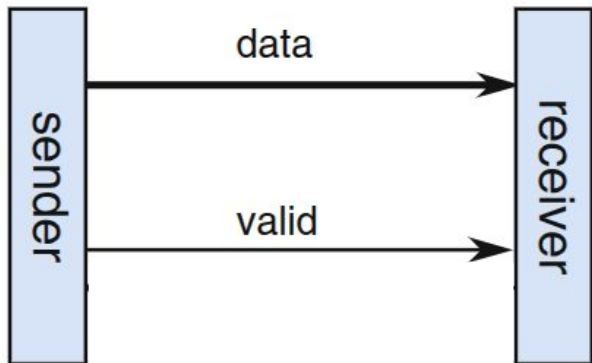


Source-synchronous protocol



- **Idle** – передатчик не готов передавать данные ($\text{valid}=0$)
- **Fire** – передатчик передает, а приемник принимает данные ($\text{valid}=1$)

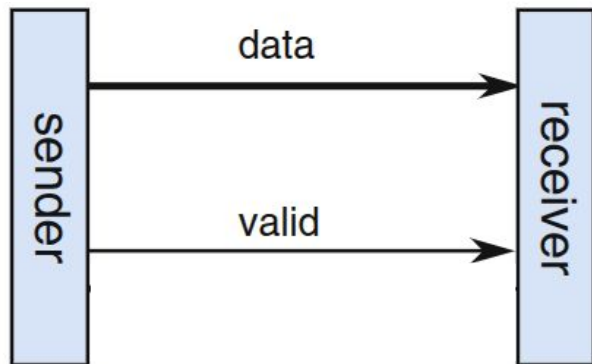
Source-synchronous protocol



- Если отправитель хочет передать данные, получатель **обязан** их принять

- **Idle** – передатчик не готов передавать данные ($\text{valid}=0$)
- **Fire** – передатчик передает, а приемник принимает данные ($\text{valid}=1$)

Source-synchronous protocol



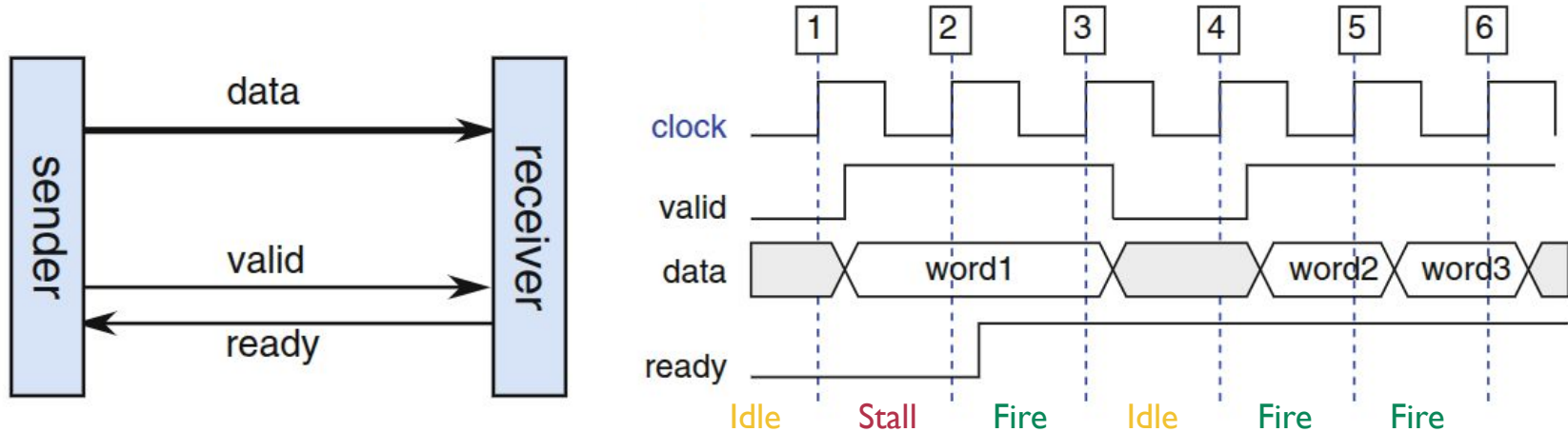
- Если отправитель хочет передать данные, получатель **обязан** их принять
- Это условие не всегда выполнимо
- Например, пропускная способность внешних интерфейсов (UART/I2C/SPI/USB/DDR/PCIe) ограничена

- **Idle** – передатчик не готов передавать данные (valid=0)
- **Fire** – передатчик передает, а приемник принимает данные (valid=1)

Flow control

Управление потоком (flow control) – механизм регулирования скорости передачи данных в сетях для предотвращения перегрузки приемника, когда он не успевает обрабатывать информацию, поступающую от более быстрого отправителя

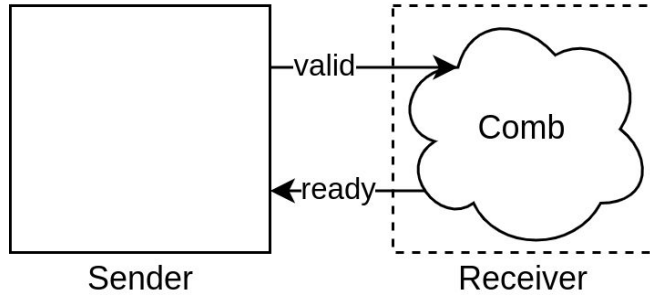
Valid-ready handshake



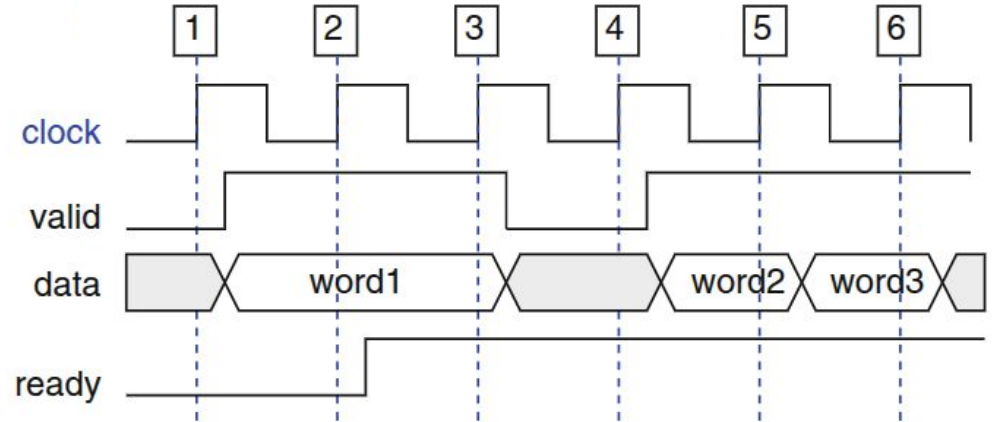
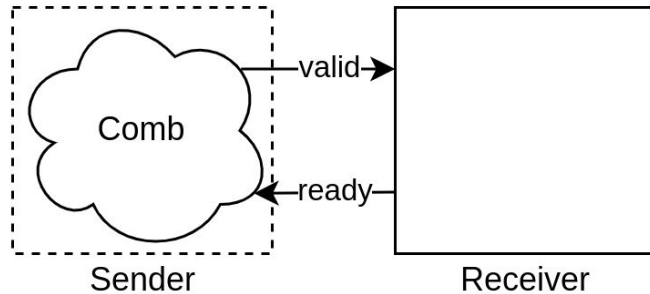
- **Idle** – передатчик не готов передавать данные ($\text{valid}=0, \text{ready}=0/1$)
- **Transfer (Fire)** – передатчик передает, а приемник принимает данные ($\text{valid}=1, \text{ready}=1$)
- **Stall (Back-pressure)** – приемник не готов принимать данные ($\text{valid}=1, \text{ready}=0$)

Valid-ready handshake

Valid-then-ready

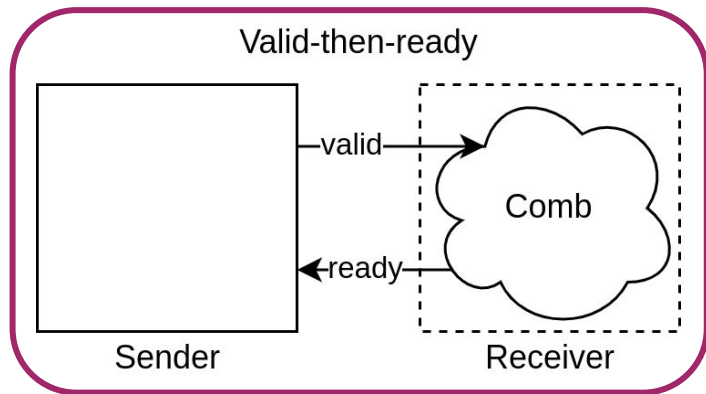


Ready-then-valid

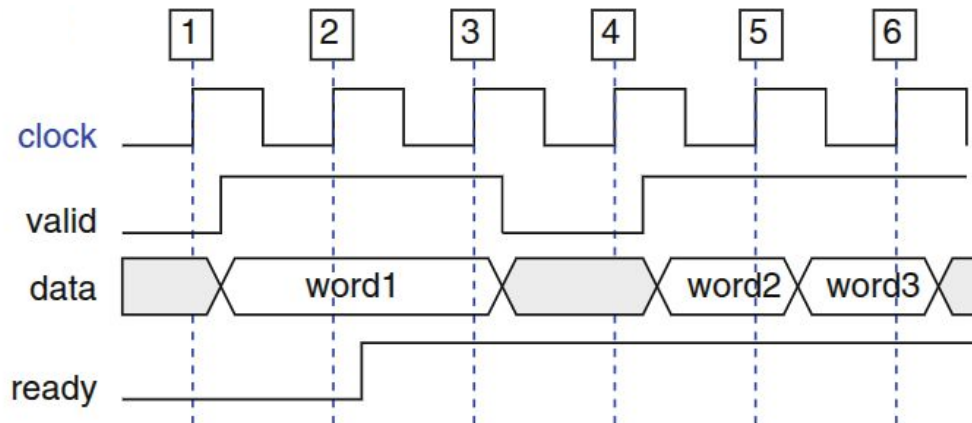
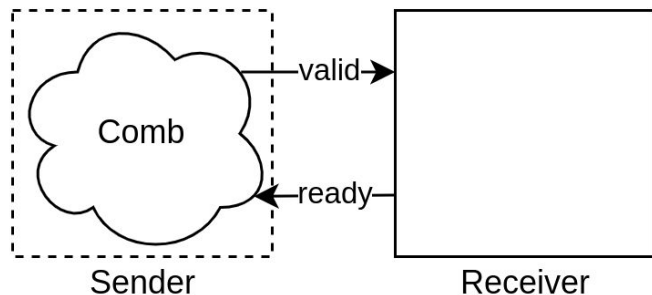


Valid-ready handshake

Valid-then-ready

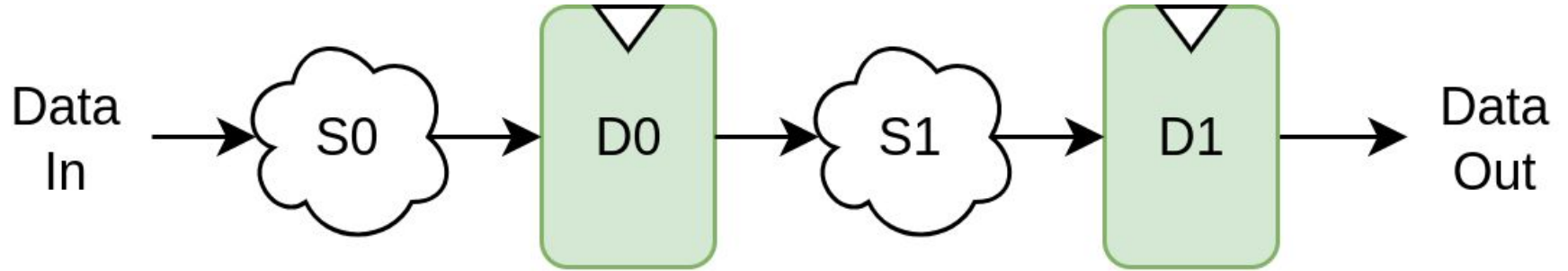


Ready-then-valid

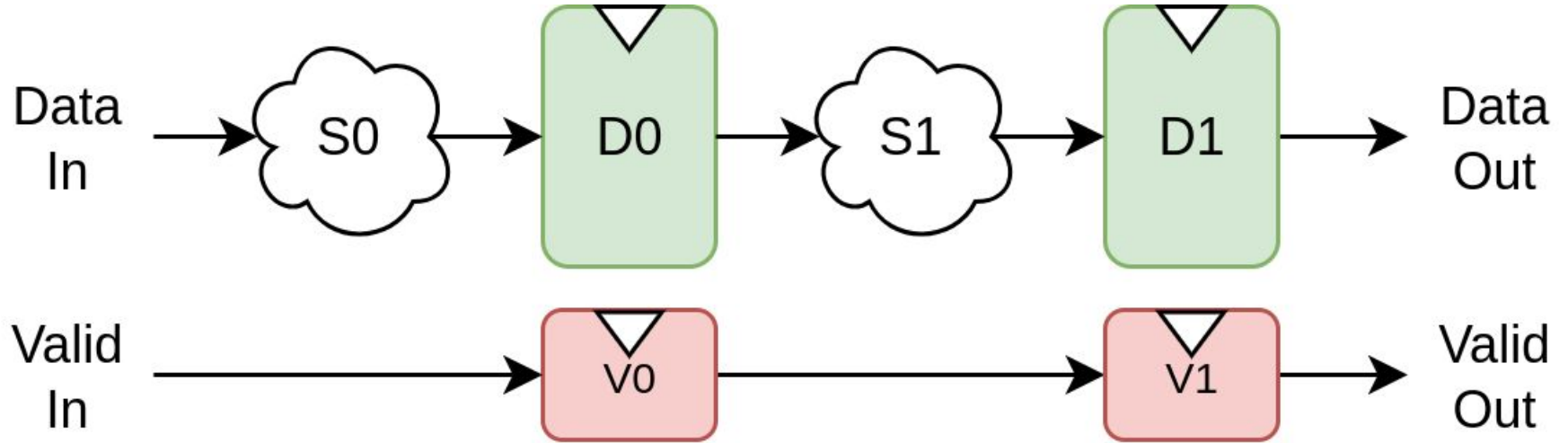


Как правило, в протоколах на основе **valid-then-ready handshake**, от приемника требуется **не снимать valid** и **не менять данные**, пока не произошла передача. Также, **valid должен быть 0 в состоянии reset**. Такие требования, например, предъявляют все версии AXI, ACE, AXI-Stream.

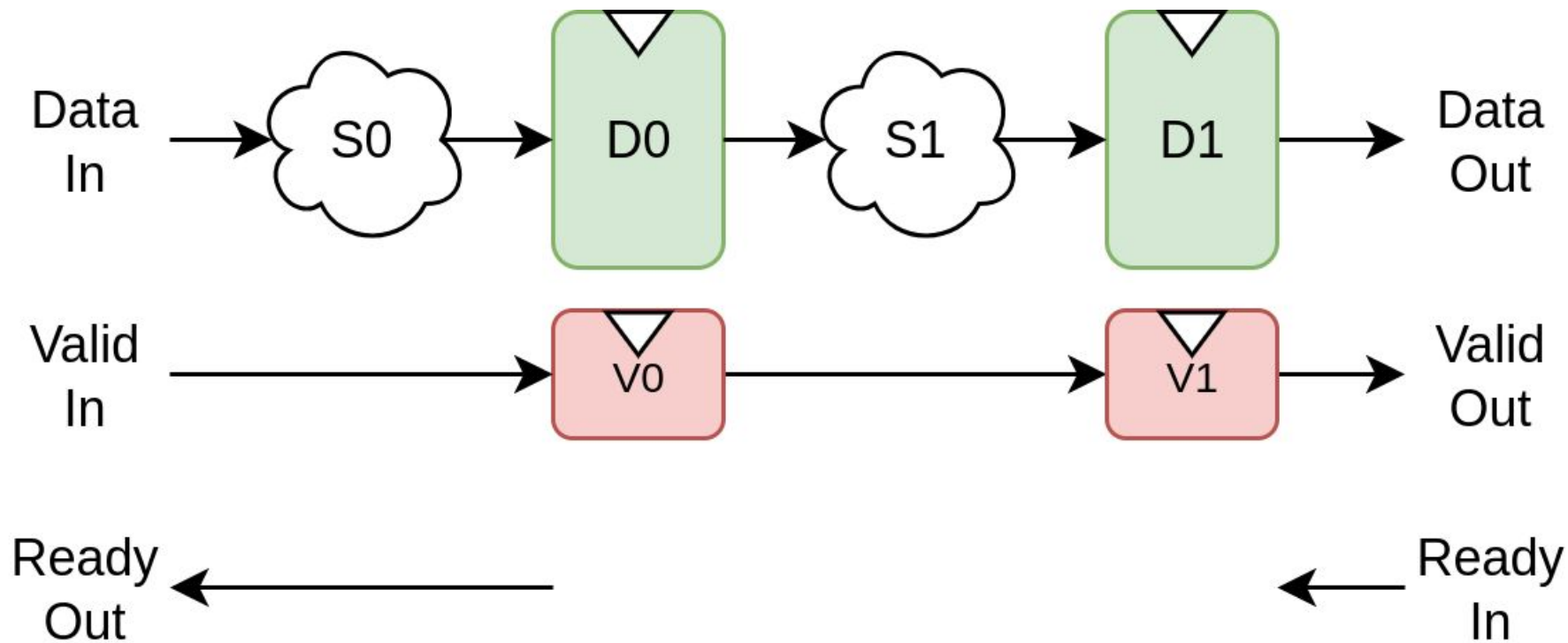
Pipeline



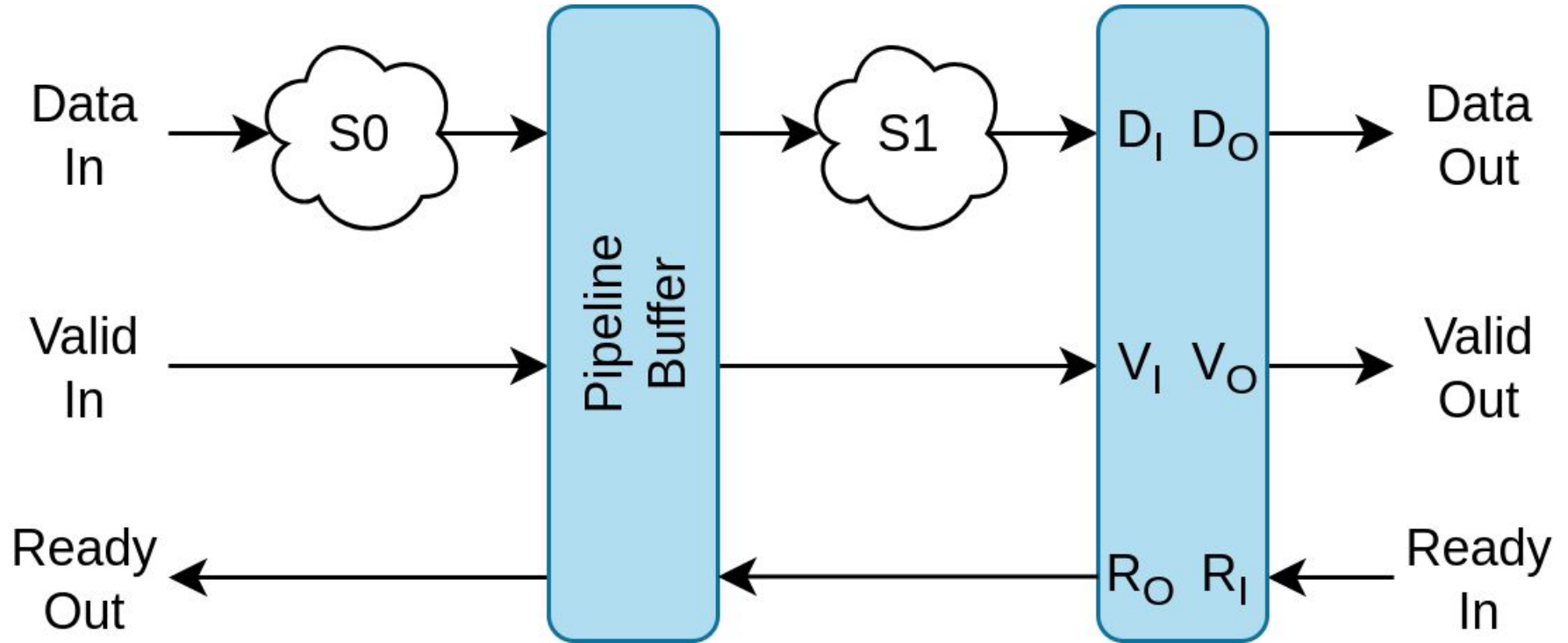
Pipeline



Pipeline



Pipeline



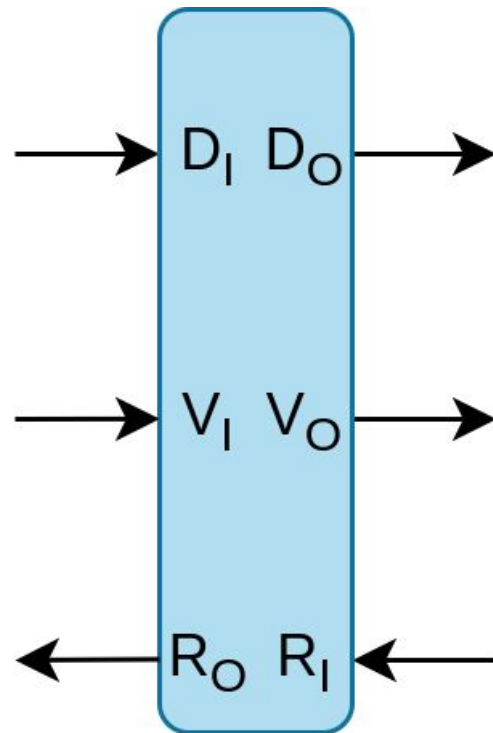
Pipeline buffer

Начальное состояние:

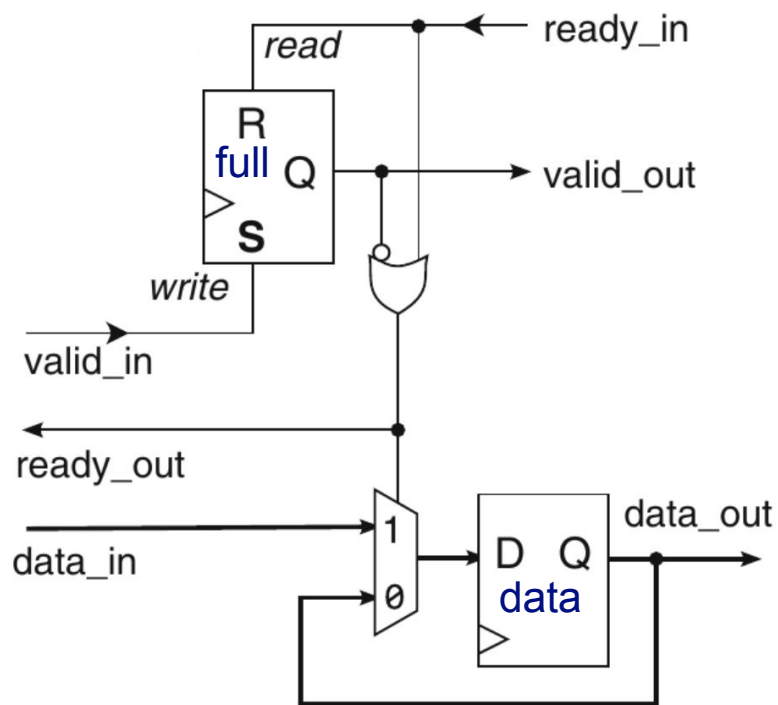
- Буфер пуст
- Если $R_{in}=1$ или буфер пустой, $R_{out}=1$, иначе $R_{out}=0$
- Если буфер полный, то $V_{out}=0$, иначе $V_{out}=1$

На каждый следующий такт:

- Если $V_{in}=1$, $R_{in}=X$, то буфер станет полным
- Если $V_{in}=0$, $R_{in}=1$, то буфер станет пустым
- Если $V_{in}=0$, $R_{in}=0$, то буфер останется пустым/полным
- Если $R_{in}=1$, данные со входа будут сохранены в буфер
- Если буфер пуст, данные со входа будут сохранены в буфер



Elastic buffer: full-throughput (pipeline)



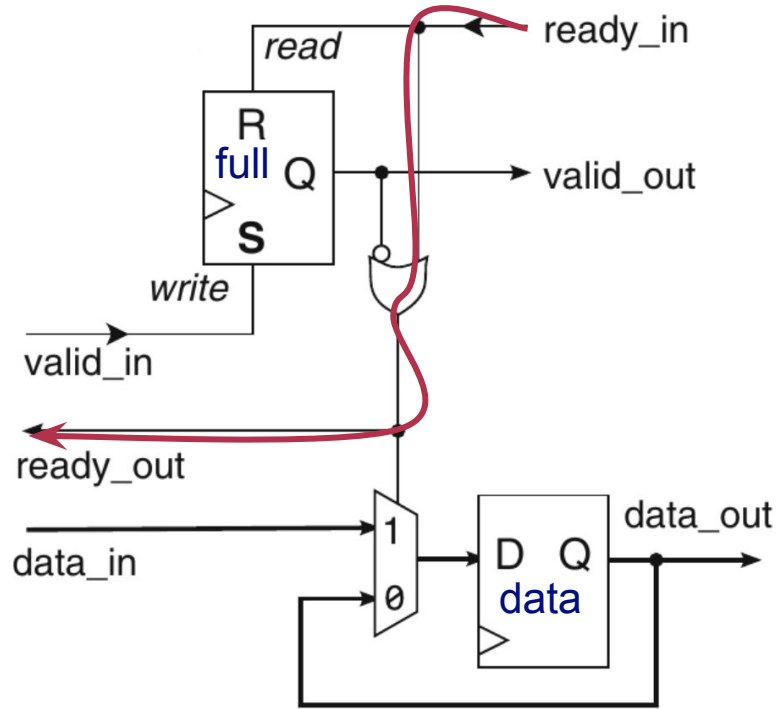
```
reg [WIDTH-1:0] data;
reg full;

wire valid_out = full;
wire ready_out = !full || ready_in;
wire [WIDTH-1:0] data_out = data;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        full <= 1'b0;
    else
        if (valid_in)
            full <= 1'b1;
        else if (ready_in)
            full <= 1'b0;
end

always @(posedge clk) begin
    if (!full || ready_in)
        data <= data_in;
end
```

Elastic buffer: full-throughput (pipeline)



```
reg [WIDTH-1:0] data;
reg full;

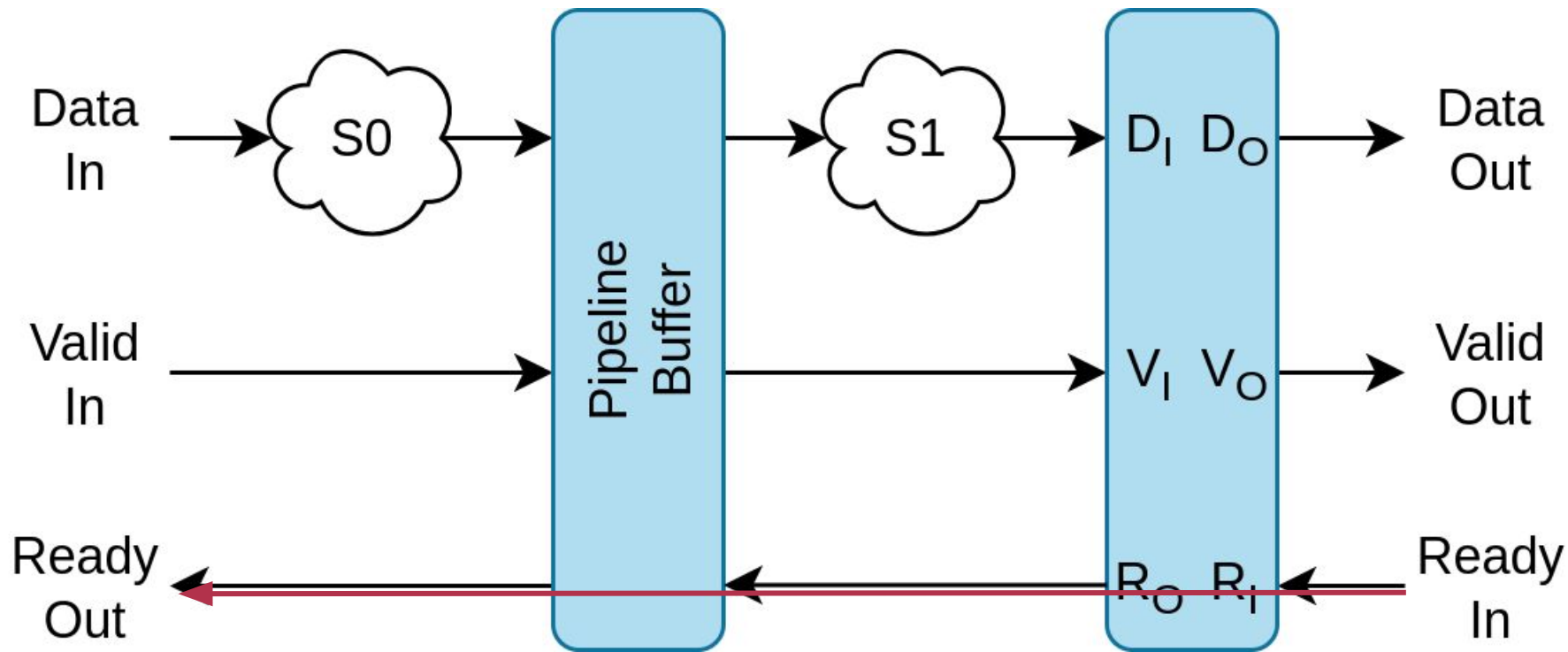
wire valid_out = full;
wire ready_out = !full || ready_in;
wire [WIDTH-1:0] data_out = data;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        full <= 1'b0;
    else
        if (valid_in)
            full <= 1'b1;
        else if (ready_in)
            full <= 1'b0;
end

always @(posedge clk) begin
    if (!full || ready_in)
        data <= data_in;
end
```

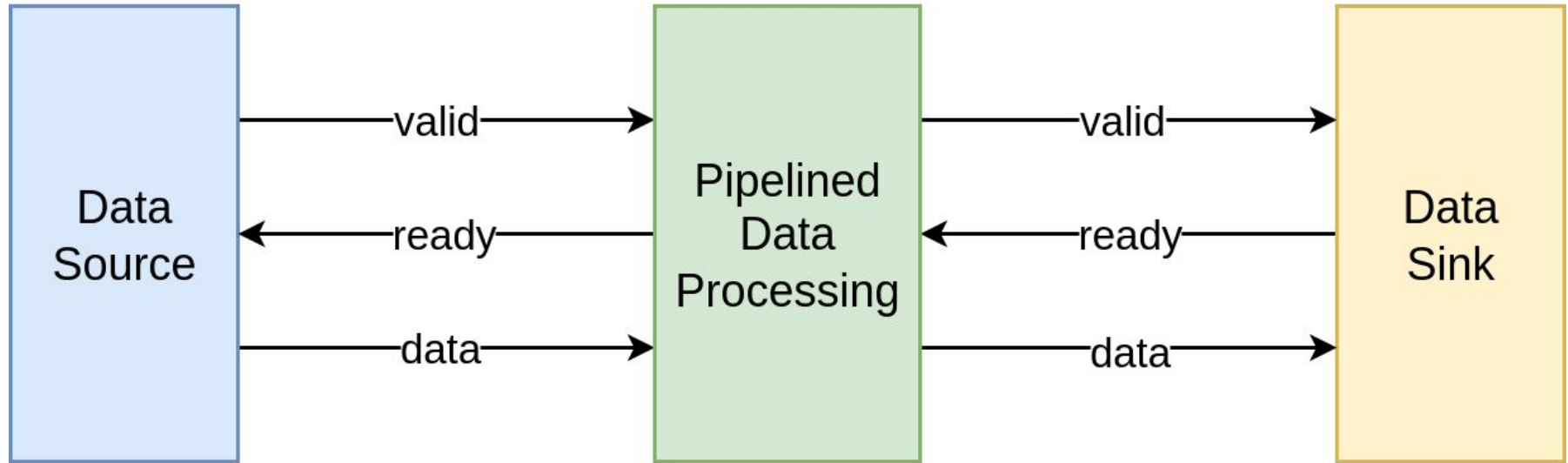
Входной и выходной ready **связаны комбинационно**

Pipeline

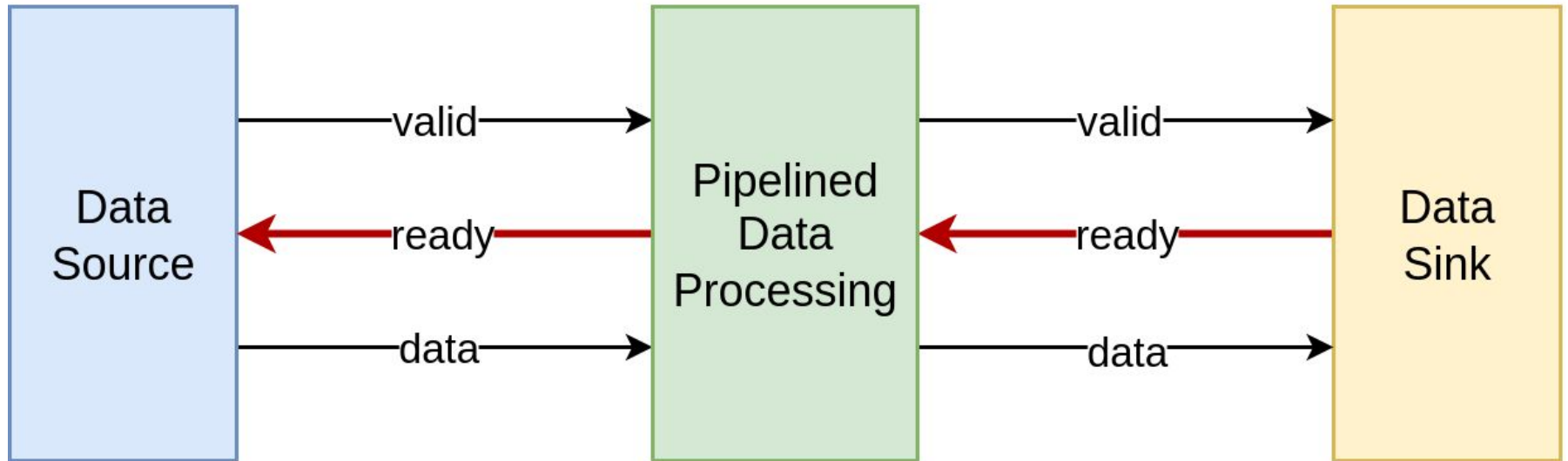


Входной и выходной ready связаны комбинационно $\Rightarrow$ есть ограничение на число стадий

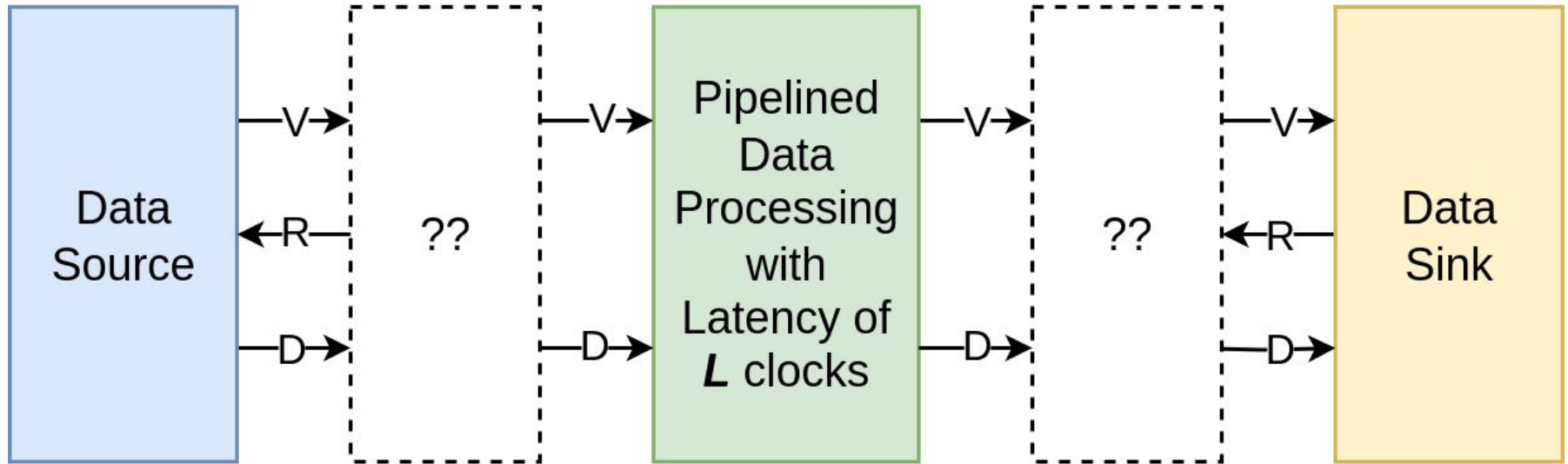
Valid/ready flow control



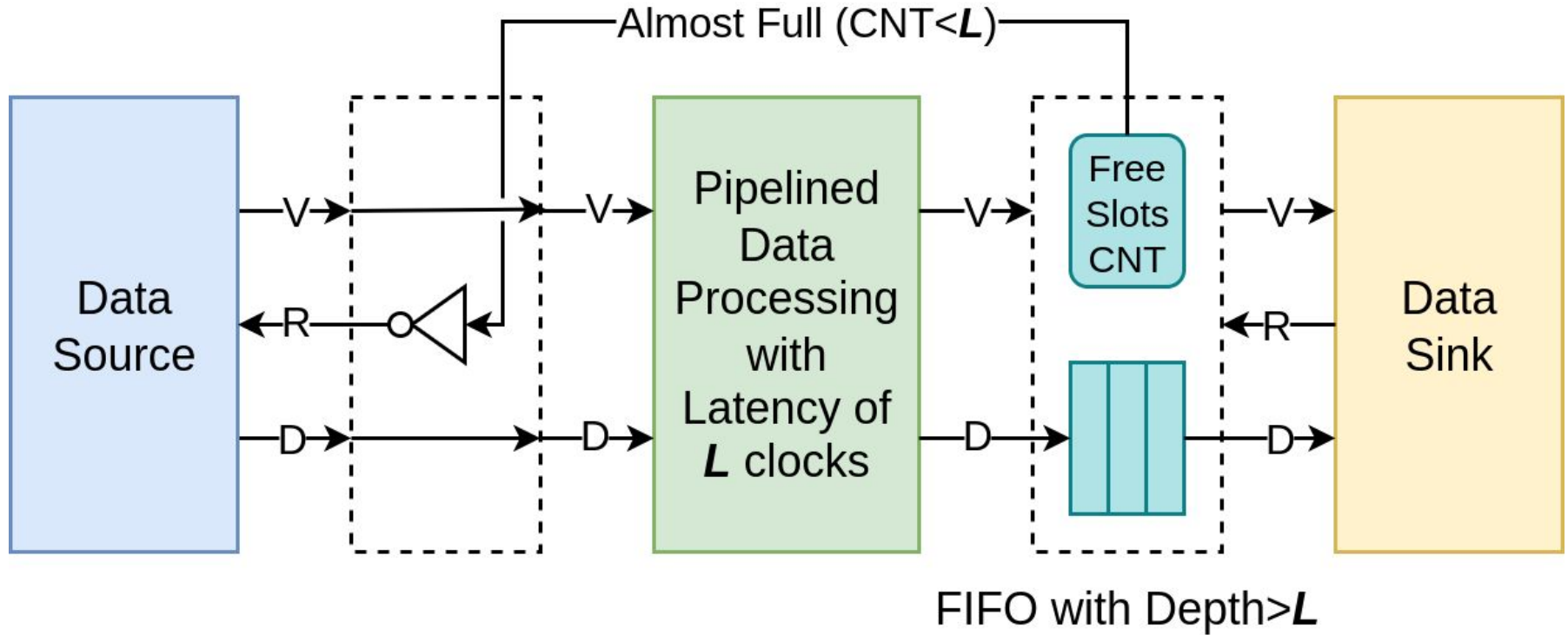
Valid/ready flow control



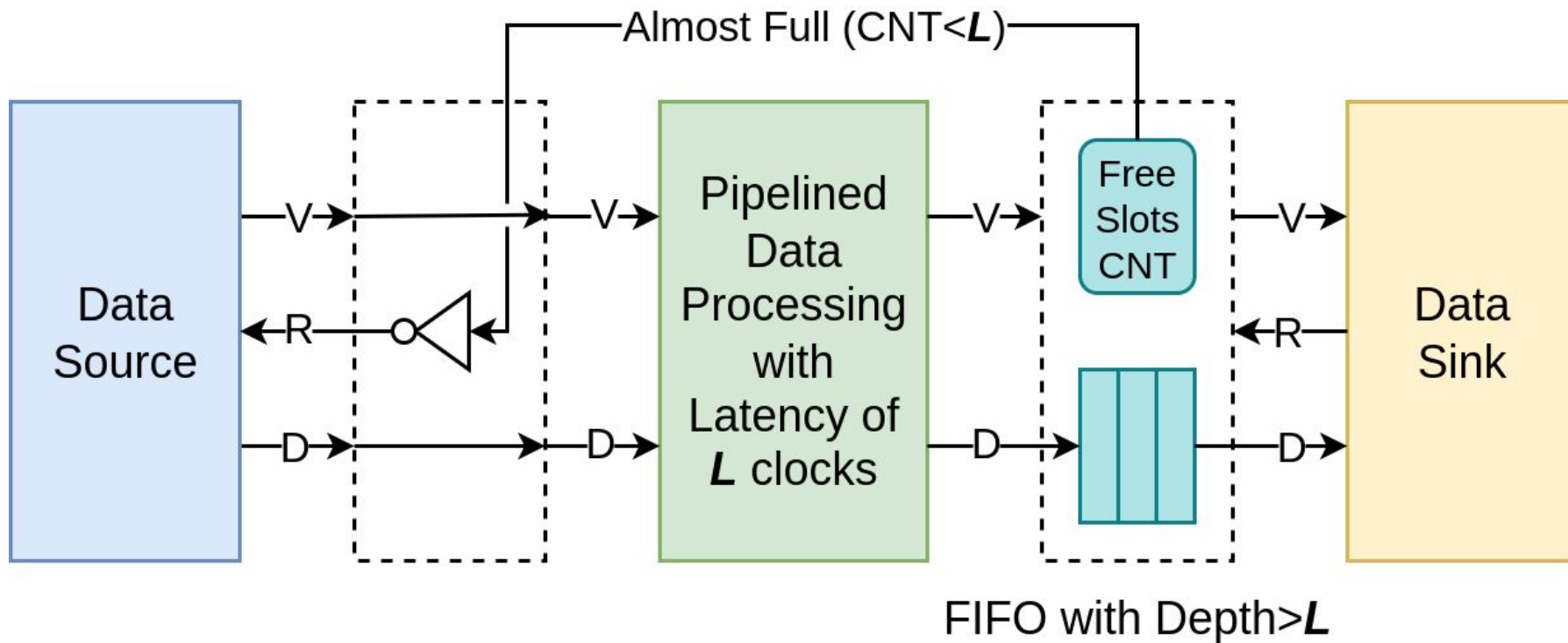
Valid/ready flow control



FIFO with Almost-Full

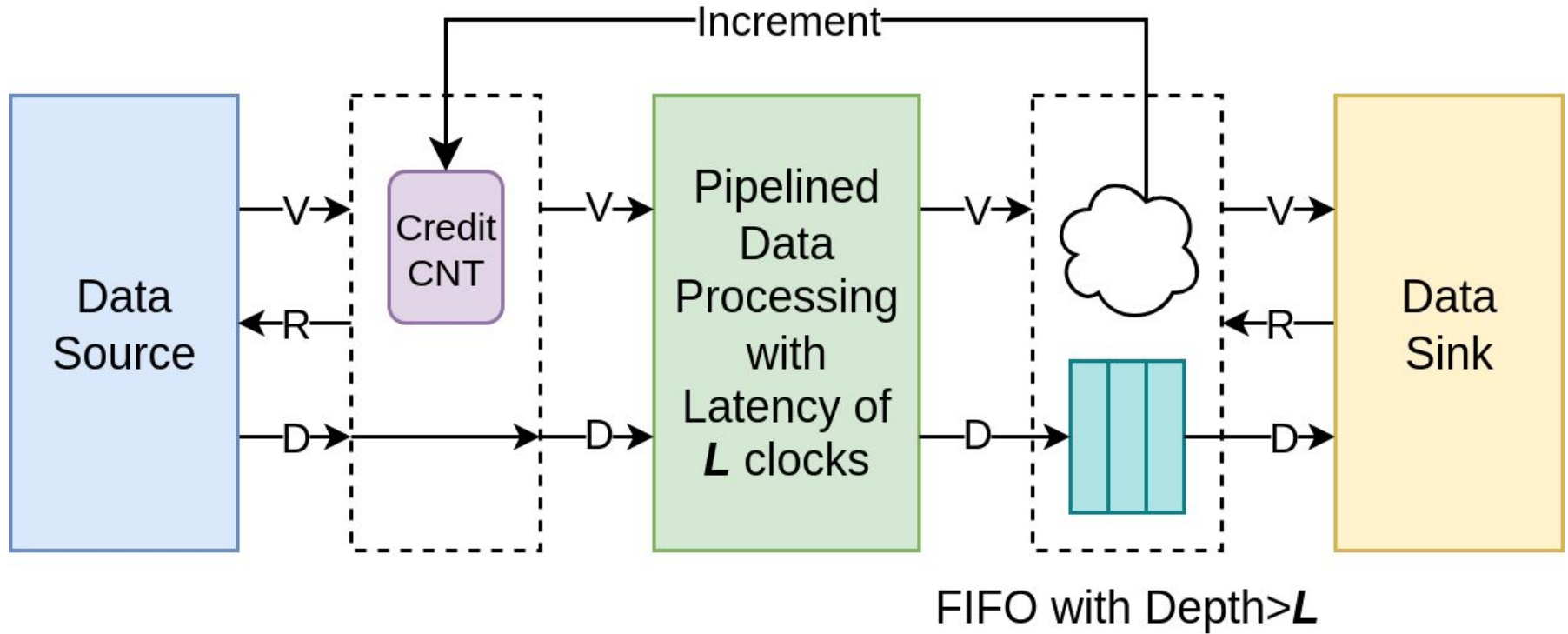


FIFO with Almost-Full

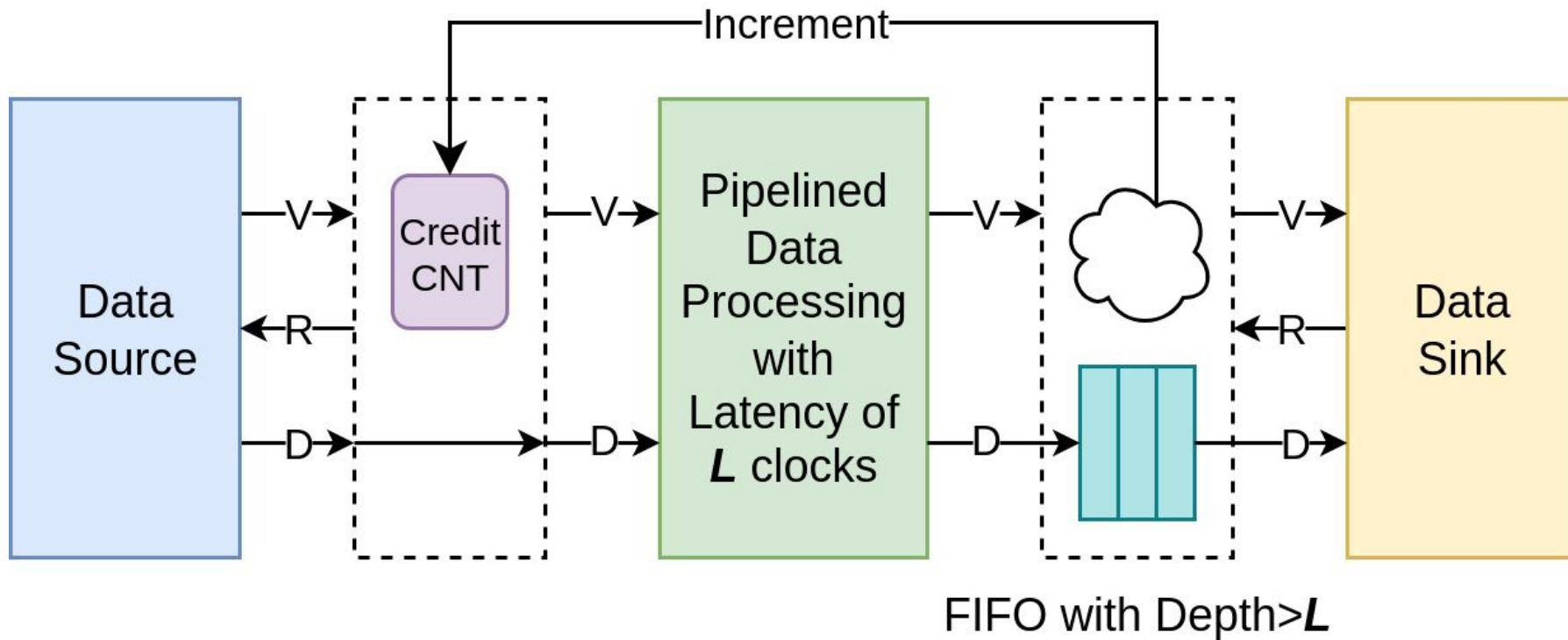


- Сигнал Almost-Full не позволяет передатчику переполнить очередь на приемной стороне

Credit-based flow control

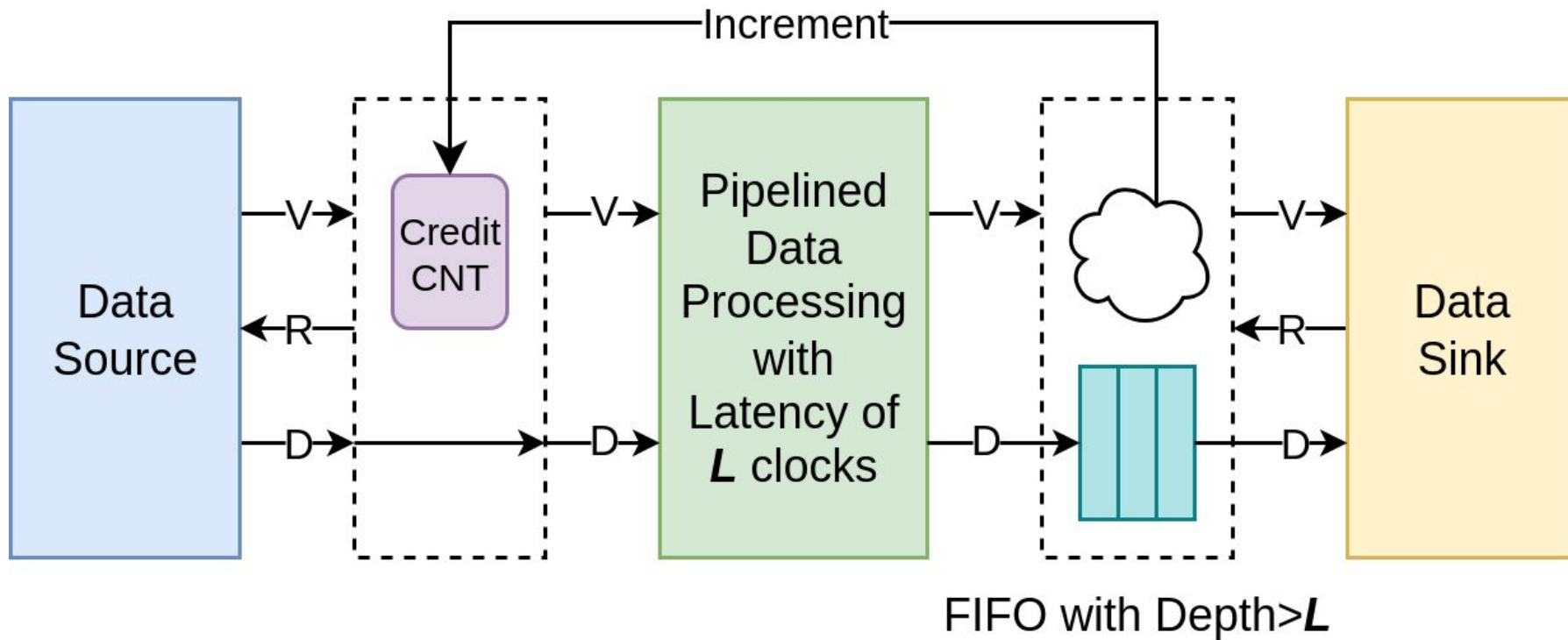


Credit-based flow control



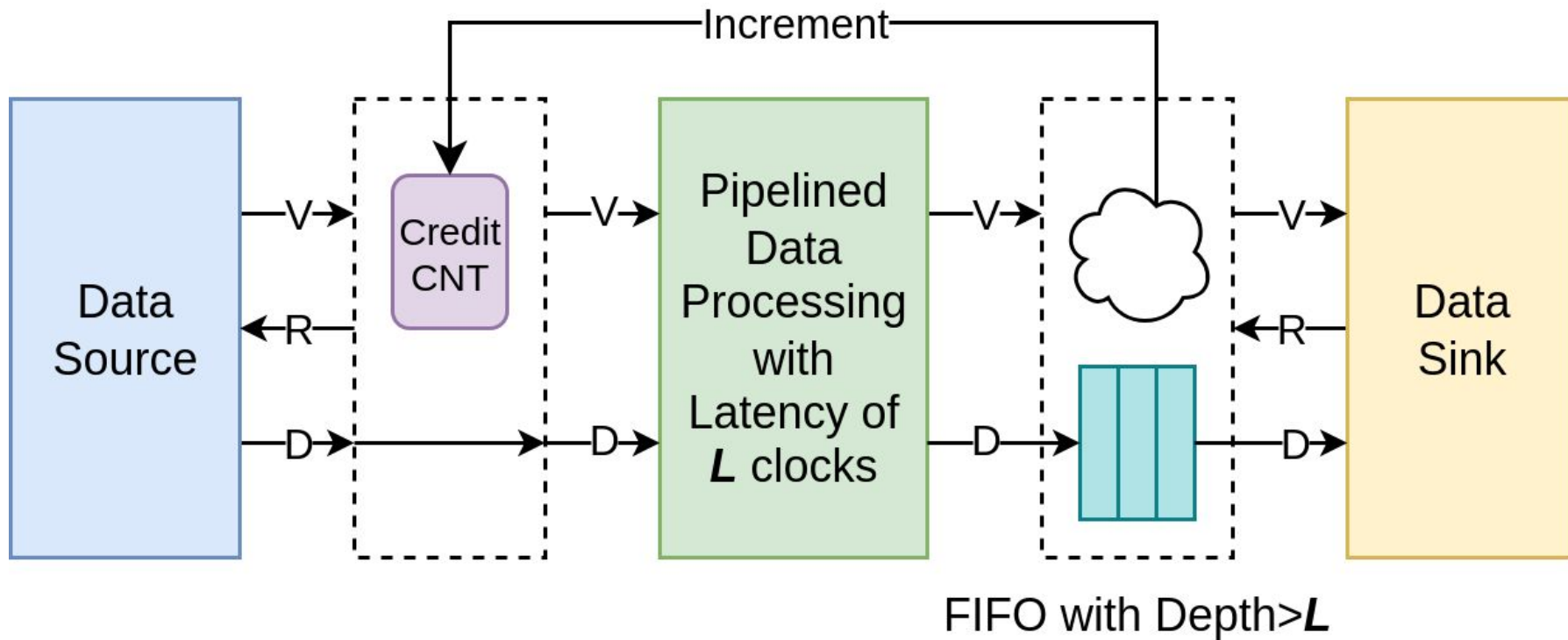
- Основная идея – перенос счетчика с приемной стороны на передающую
- Начальное значение кредитного счетчика – размер очереди на приемной стороне

Credit-based flow control



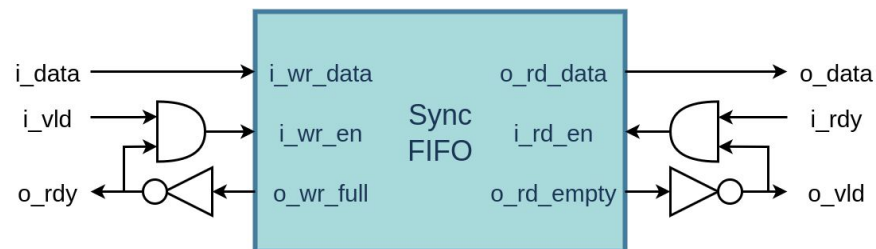
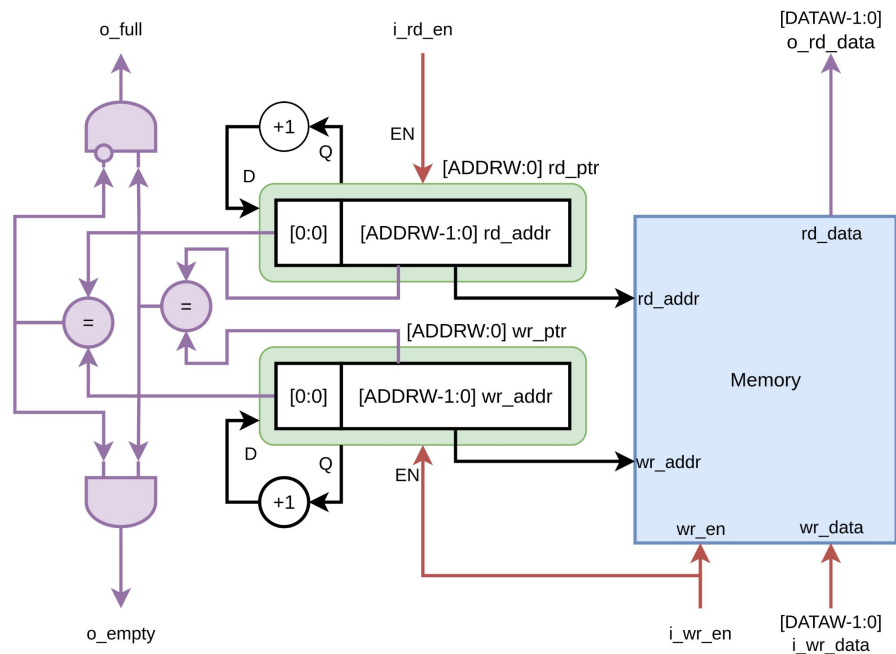
- Кредитный счетчик **уменьшается на 1** при каждой передаче от источника, пока не достигнет 0
- При нулевом количестве кредитов передача запрещена

Credit-based flow control

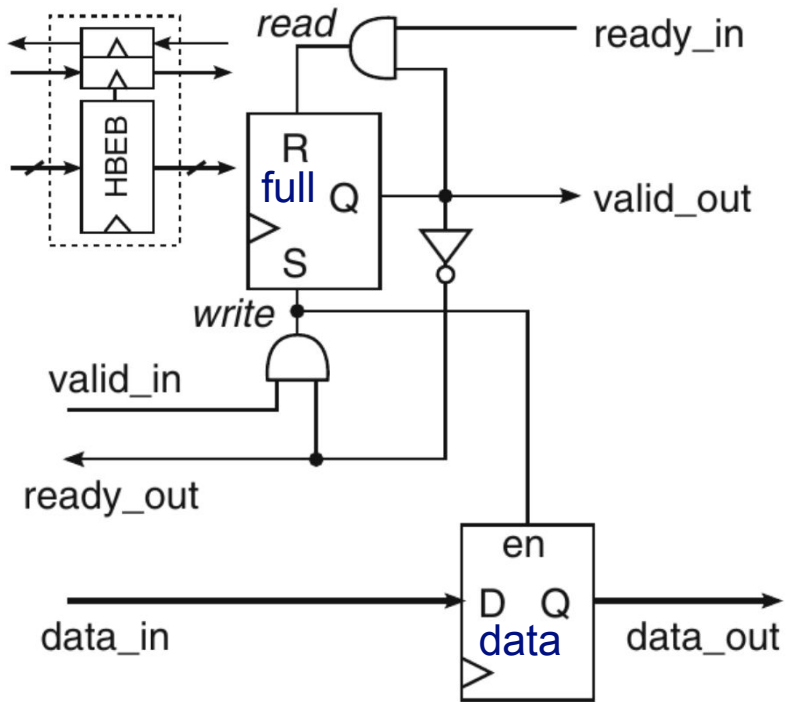


- Кредитный счетчик **увеличивается на 1** (до макс. значения) при каждой передаче к приемнику
- Увеличение и уменьшение может произойти в один такт, тогда значение не изменяется

Синхронное FIFO в роли буфера



Elastic buffer: half-throughput



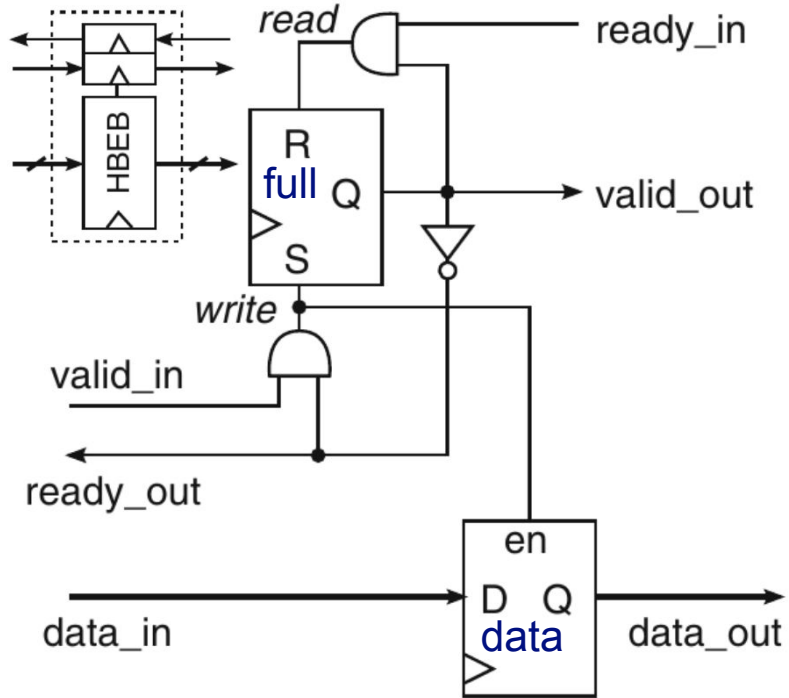
```
reg [WIDTH-1:0] data;
reg full;

wire valid_out = full;
wire ready_out = !full;
wire [WIDTH-1:0] data_out = data;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        full <= 1'b0;
    else
        if (valid_in && !full)
            full <= 1'b1;
        else if (ready_in && full)
            full <= 1'b0;
end

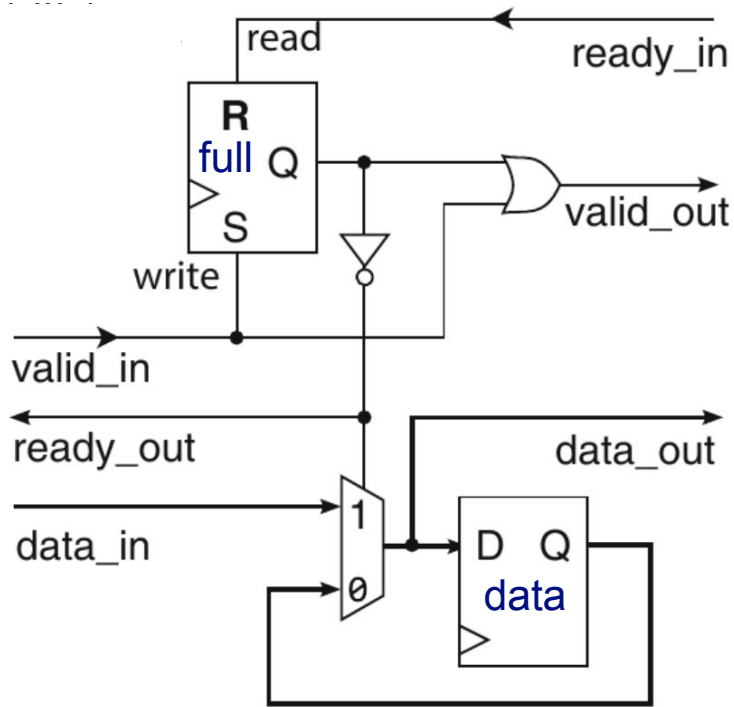
always @(posedge clk) begin
    if (valid_in && !full)
        data <= data_in;
end
```

Elastic buffer: half-throughput



cycle		0	1	2	3	4	5	6
Write Side	r	1	0	1	0	0	1	0
	v	1	1	1	1	1	1	1
	D	A	B	B	C	C	C	D
EB		*	A	*	B	B	*	C
Read Side	r	1	1	1	0	1	1	1
	v	0	1	0	1	1	0	1
	D	*	A	*	B	B	*	C

Elastic buffer: full-throughput (bypass)



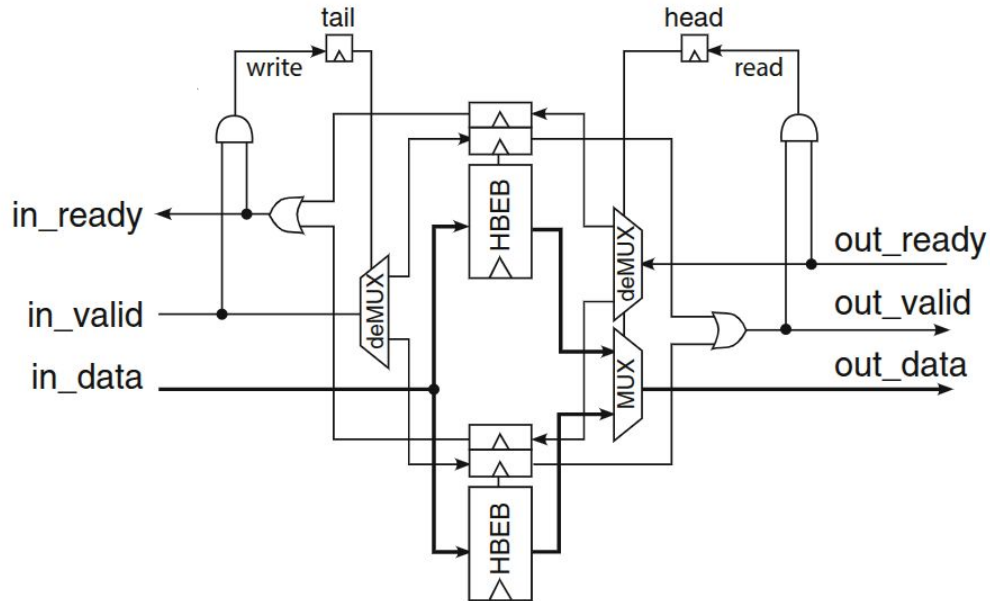
```
reg [WIDTH-1:0] data;
reg full;

wire valid_out = full || valid_in;
wire ready_out = !full;
wire [WIDTH-1:0] data_out = full ? data : data_in;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        full <= 1'b0;
    else
        if (ready_in)
            full <= 1'b0;
        else if (valid_in)
            full <= 1'b1;
end

always @(posedge clk) begin
    if (!full)
        data <= data_in;
end
```

Elastic buffer: full-throughput (2-slot)

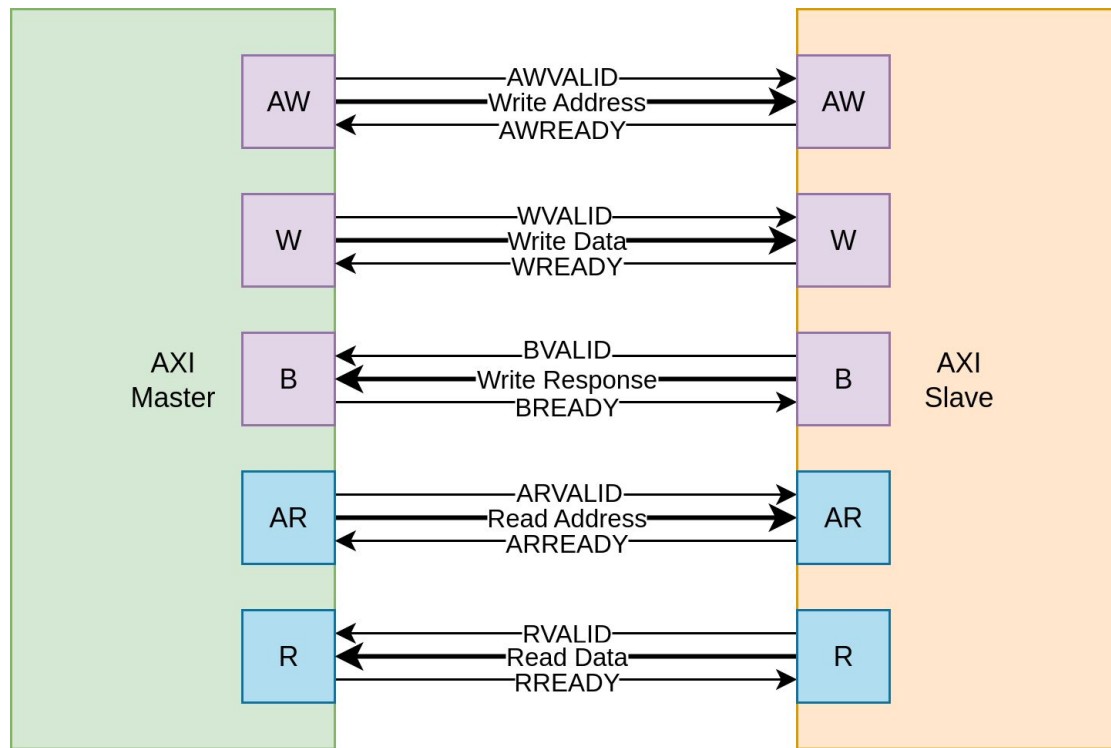


cycle		0	1	2	3	4	5	6
Write Side	r	1	1	1	1	0	1	1
	v	1	1	1	1	1	1	1
	D	A	B	C	D	E	E	F
EB #0		*	A	*	C	C	*	E
EB #1		*	*	B	*	D	D	*
Read Side	r	1	1	1	0	1	1	1
	v	0	1	1	1	1	1	1
	D	*	A	B	C	C	D	E

Elastic buffer

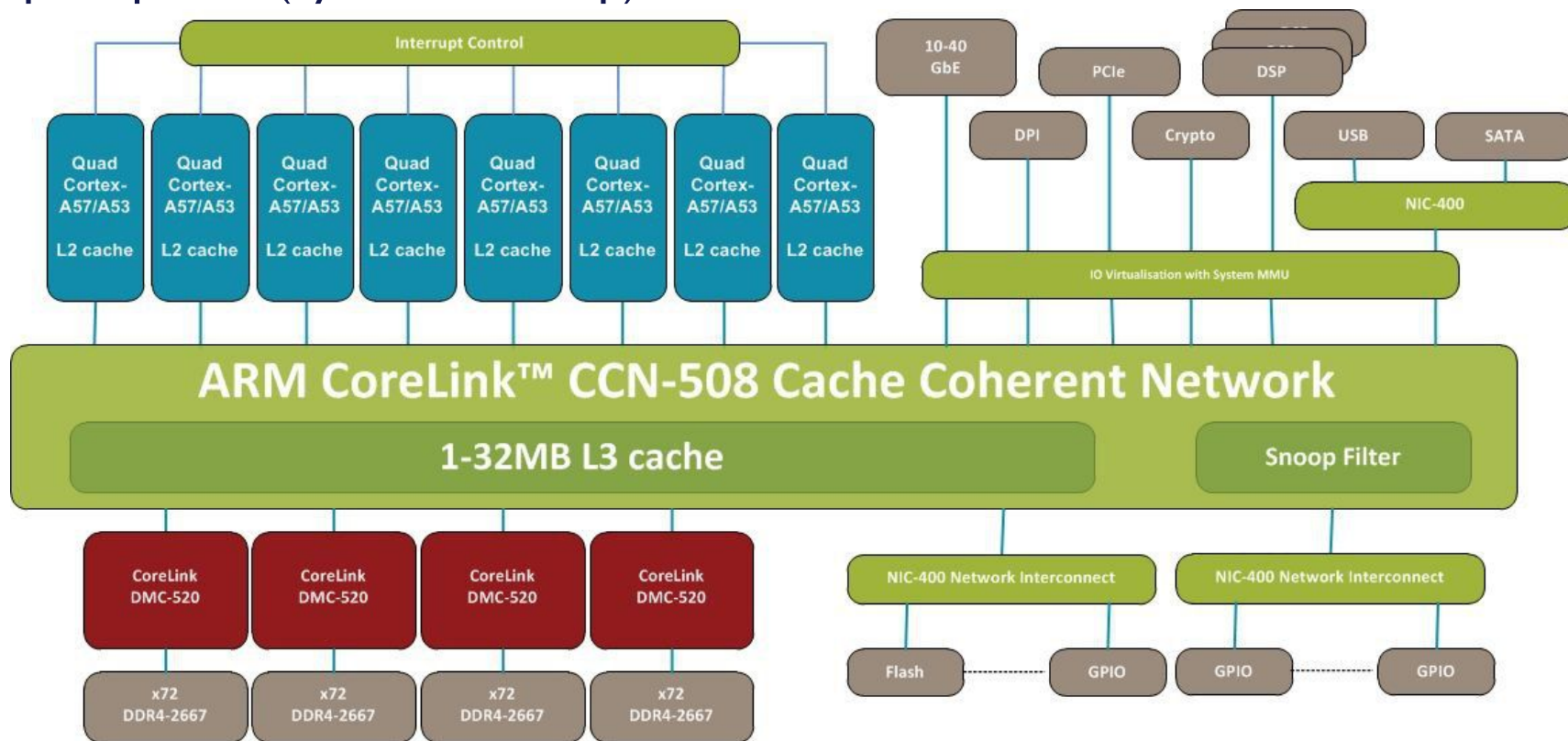
		Valid и Data	
		Связаны комбинационно	Не связаны комбинационно
Ready	Связаны комбинационно	Passthrough Latency = 0 0 bits	Pipeline Latency of 1 WIDTH+1 bits
	Не связаны комбинационно	Bypass Latency = 0 WIDTH+1 bits	Two-slot Latency = 1 2*WIDTH+4 bits

Пример: AXI



AXI – пример интерфейса на основе valid/ready flow control

Пример SoC (System-on-Chip)



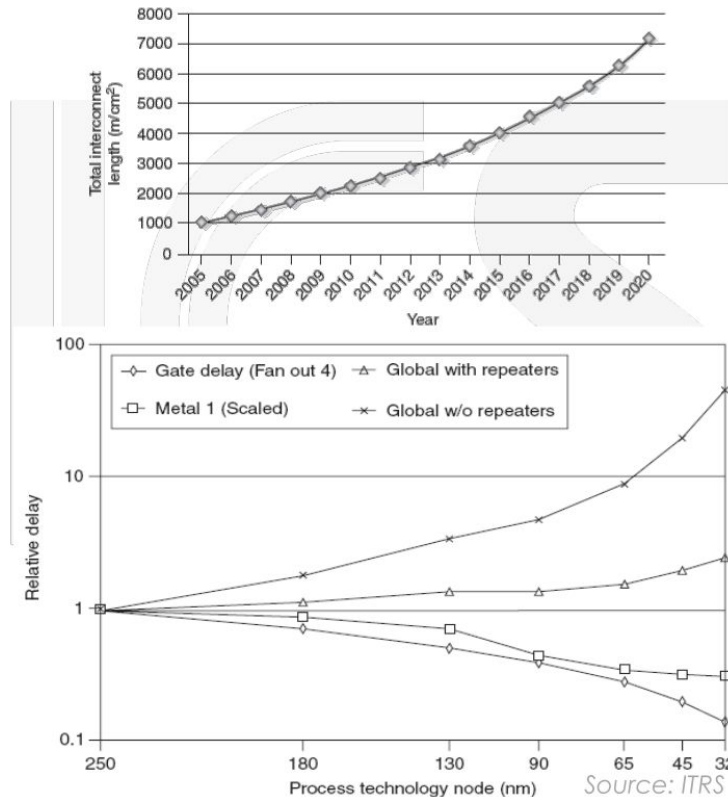
Масштабирование

Характеристики глобальных соединений на кристалле масштабируются **хуже** чем характеристики логических ячеек

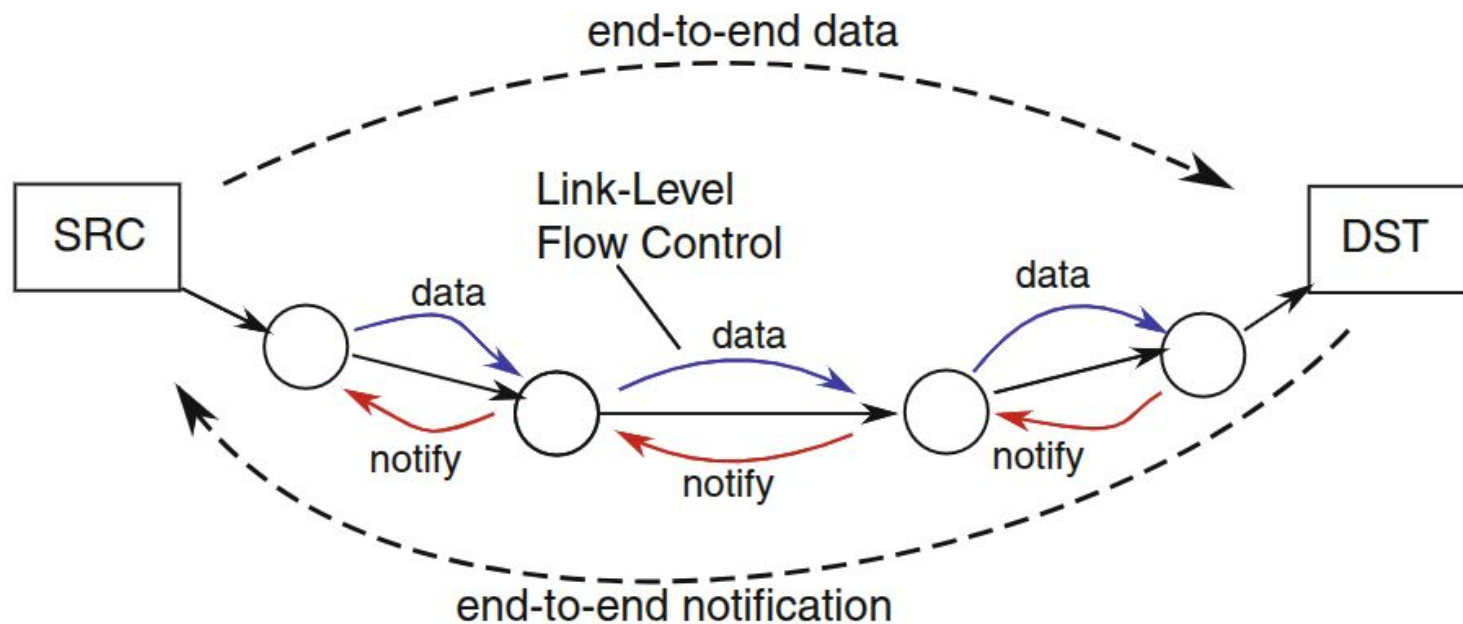
- Свойства ячеек улучшаются при смене технологии
- Отношение задержки передачи к задержке вычислений изменяется с 2:1 к 9:1 при переходе от 130nm к 50nm

Operation	Delay	
	(0.13um)	(0.05um)
32b ALU Operation	650ps	250ps
32b Register Read	325ps	125ps
Read 32b from 8KB RAM	780ps	300ps
Transfer 32b across chip (10mm)	1400ps	2300ps
Transfer 32b across chip (20mm)	2800ps	4600ps

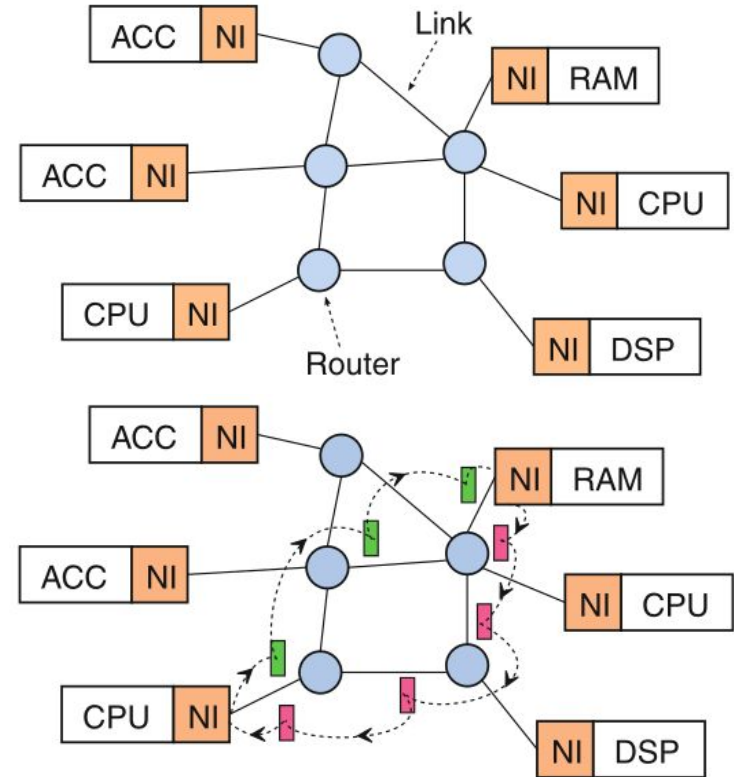
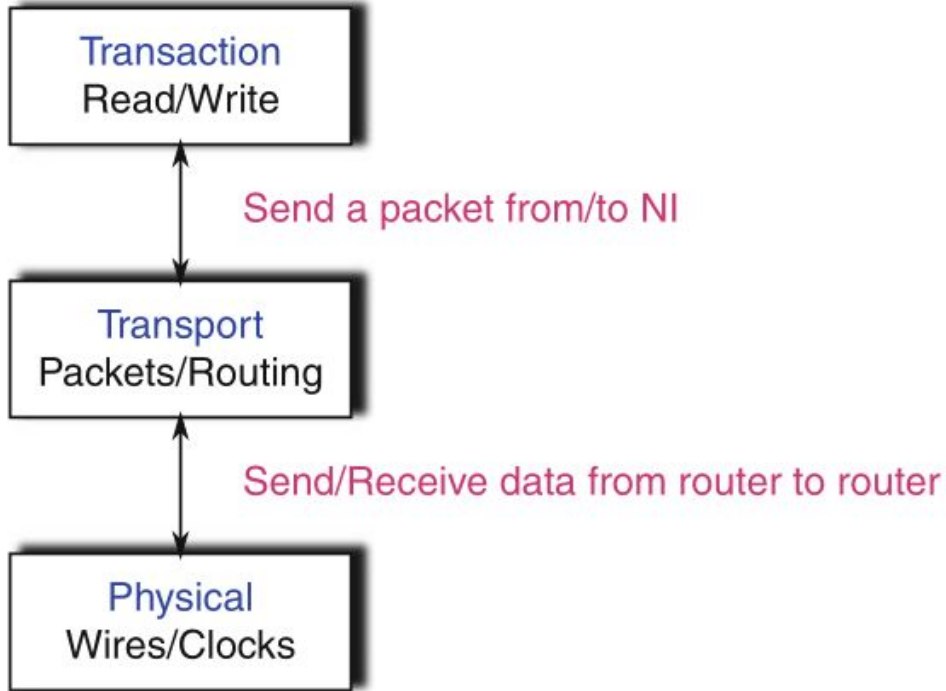
Source: Bill Dally, DAC 2009 keynote



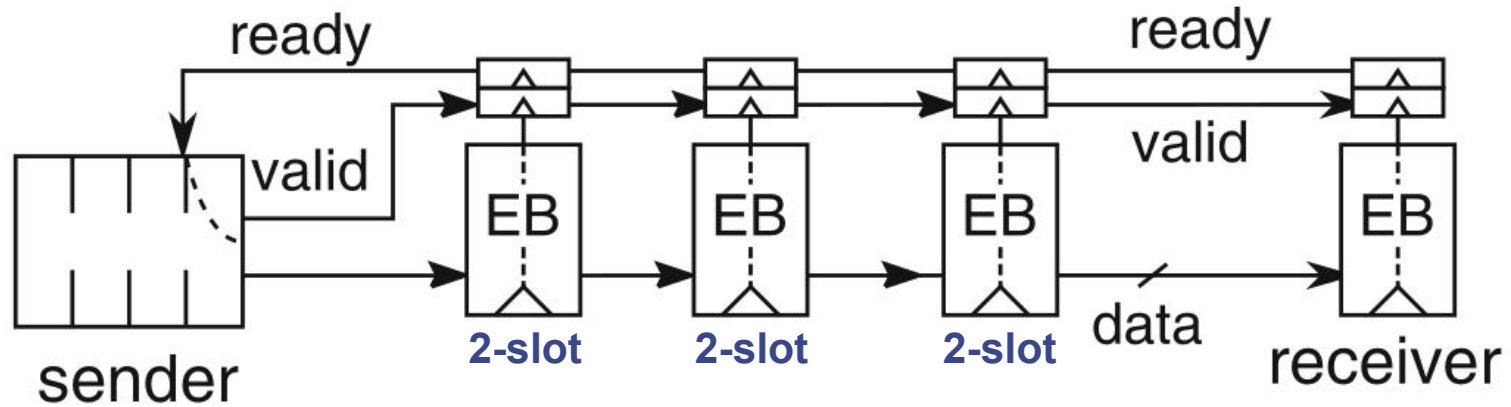
Поток данных в NoC



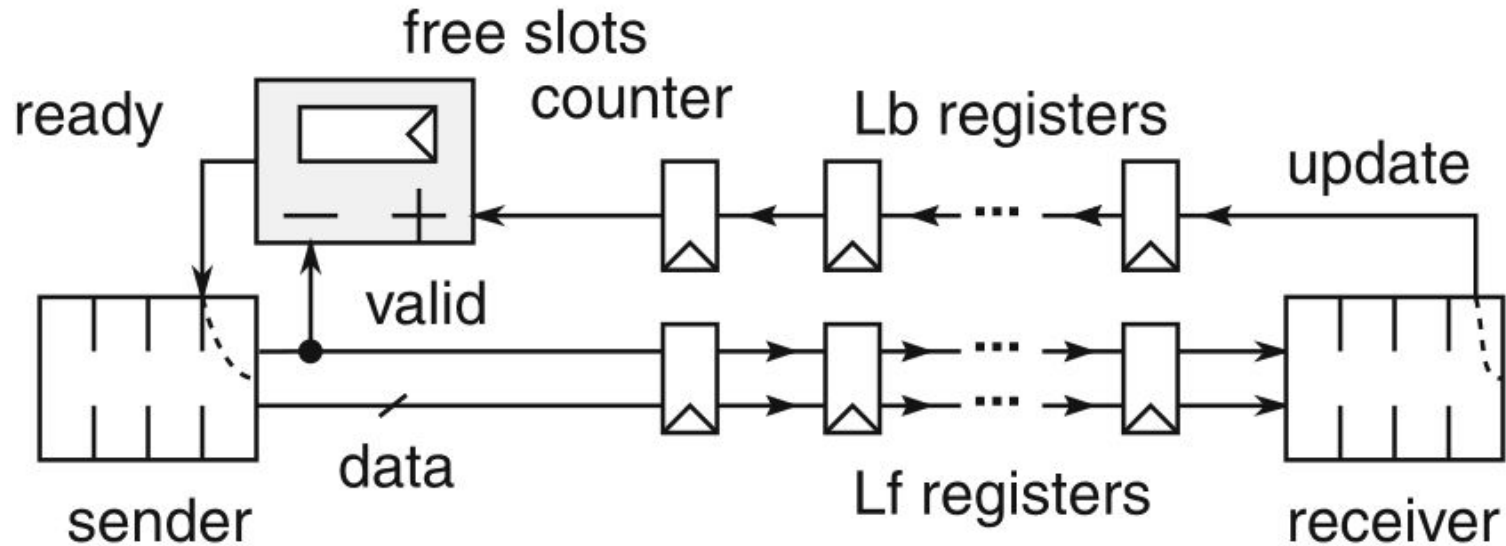
Network-on-Chip



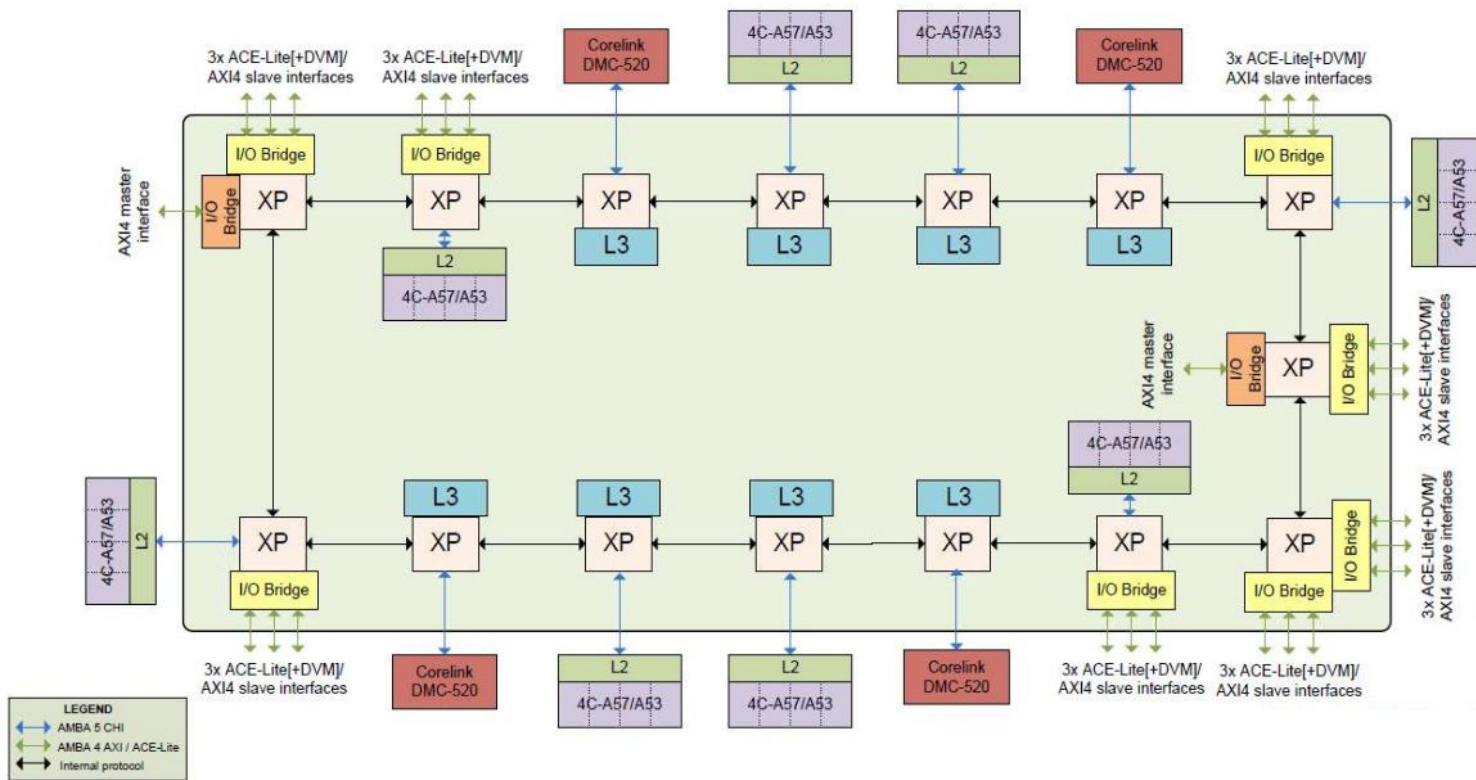
Pipelined Links with Elastic Buffers



Pipelined Links with Credit-Based Control Flow



Пример NoC (Network-on-Chip)



Задание

- Поддержать valid/ready flow control в систолическом массиве с помощью FIFO и credit counters

Литература

- G. Dimitrakopoulos, A. Psarras and I. Seitanidis, “Microarchitecture of Network-on-Chip Routers: A Designer’s Perspective”
- SoC 101: On-Chip Interconnect